



[NOM DE L'ÉTABLISSEMENT]

[Filière — ex. Génie Informatique / Ingénierie Logicielle]

Rapport de Projet de Fin d'Études

IAT Academy

Conception et réalisation d'une plateforme e-learning pour les métiers
de l'aviation, de l'accueil et du tourisme



Réalisé par :

[Prénom NOM]

Encadré par :

[Encadrant académique — Prénom
NOM]

[Encadrant professionnel —
Prénom NOM]

Année universitaire [2025 – 2026]

Remerciements

Je tiens à exprimer ma sincère gratitude à mon encadrant académique pour son suivi attentif tout au long de ce projet, sa disponibilité et la pertinence de ses remarques, qui ont considérablement enrichi la démarche suivie dans ce travail. Mes remerciements vont également à mon encadrant professionnel au sein d'IAT Academy, pour la confiance qu'il m'a accordée dès le début du stage en me confiant un périmètre technique aussi complet, et pour le temps consacré à clarifier les règles métier propres au cursus de l'académie — sans lesquelles la conception du système de progression par unité de formation n'aurait pas pu aboutir à une solution fidèle à la réalité du terrain.

Je remercie chaleureusement l'ensemble de l'équipe pédagogique et administrative d'IAT Academy pour son accueil, sa bienveillance et sa disponibilité à répondre à mes questions tout au long de l'immersion dans le fonctionnement de l'académie. Cette collaboration au quotidien a été déterminante pour comprendre non seulement les besoins fonctionnels exprimés, mais aussi les contraintes réelles d'exploitation d'une plateforme destinée à des apprenants et des formateurs aux profils variés.

Je souhaite également remercier le corps professoral de mon établissement pour la formation théorique et pratique reçue tout au long de mon cursus, qui a constitué le socle indispensable à la réalisation de ce projet, ainsi que les membres du jury pour l'intérêt qu'ils porteront à ce travail.

Enfin, mes remerciements les plus personnels vont à ma famille et à mes proches, pour leur soutien constant et leurs encouragements tout au long de cette période de stage.

Résumé

IAT Academy est une plateforme e-learning destinée aux métiers de l’aviation, de l’accueil et du tourisme, développée avec Spring Boot (backend) et Next.js (frontend). Ce rapport présente le contexte du projet, l’analyse et la conception de la solution, les choix techniques réalisés lors de l’implémentation — en particulier les mécanismes qui dépassent le simple CRUD (correction de quiz, génération de questions par intelligence artificielle, contrôle d’accès à granularité fine, etc. — voir chapitre 3) — ainsi que la démarche de tests et de déploiement.

Mots-clés : e-learning, Spring Boot, Spring Security, PostgreSQL, Redis, intelligence artificielle, architecture en couches.

Abstract

IAT Academy is an e-learning platform for aviation, hospitality and tourism training programs, built with Spring Boot (backend) and Next.js (frontend). This report presents the project context, the analysis and design of the solution, the technical decisions made during implementation — particularly the mechanisms that go beyond simple CRUD (quiz grading, AI-assisted question generation, fine-grained access control, etc. — see Chapter 3) — and the testing and deployment approach.

Keywords : e-learning, Spring Boot, Spring Security, PostgreSQL, Redis, artificial intelligence, layered architecture.

Table des matières

Introduction générale	1
1 Contexte général du projet	3
1.1 Organisme d'accueil	3
1.2 Contexte et problématique	4
1.3 Objectifs du projet	4
1.4 Méthodologie de travail et planning	5
1.5 Conclusion	6
2 Analyse et conception	7
2.1 Acteurs et cas d'utilisation	7
2.2 Modèle de données	8
2.2.1 Un choix de modélisation délibéré : l'UF comme attribut	12
2.3 Architecture logicielle	12
2.4 Sécurité	14
2.4.1 Authentification	14
2.4.2 Autorisation	15
2.5 Choix technologiques	15
2.6 Conclusion	16
3 Réalisation	17
3.1 Vue d'ensemble des modules implémentés	17
3.2 Points forts techniques et valeur ajoutée	17
3.2.1 Génération de questions de quiz par intelligence artificielle, avec repli entre fournisseurs	17
3.2.2 Correction hybride des quiz : automatique et manuelle, avec réconcilia- tion d'état	19
3.2.3 Progression pédagogique séquentielle par UF, avec cascade vers l'éligi- bilité au certificat	20
3.2.4 Contrôle d'accès à granularité fine, au-delà du RBAC	22
3.2.5 Limitation de débit Redis, appliquée de façon transverse	23

3.2.6	Certificats vérifiables publiquement et partage LinkedIn	24
3.2.7	Recherche globale multi-source, avec confidentialité des questions de quiz	24
3.2.8	Bourse à l'emploi dérivée de la possession d'un certificat	25
3.2.9	Envoi de campagnes email asynchrone à débit maîtrisé	25
3.2.10	Détection d'inactivité (alerte précoce)	26
3.2.11	Verrou d'édition collaboratif (distinct de la progression séquentielle) . .	26
3.2.12	Tableau de synthèse	26
3.3	Limites identifiées et pistes d'amélioration	28
3.4	Conclusion	28
4	Tests et déploiement	29
4.1	Stratégie de tests	29
4.1.1	Couverture côté backend	30
4.1.2	Couverture côté frontend et tests de charge	31
4.2	Architecture de déploiement	31
4.3	Conclusion	34
	Conclusion générale	35

Table des figures

2.1	Diagramme de cas d'utilisation	8
2.2	Diagramme de classes — vue d'ensemble simplifiée de l'application	9
2.3	Diagramme de classes — structure pédagogique et groupes	10
2.4	Diagramme de classes — évaluation	11
2.5	Diagramme de classes — stage et certification	11
2.6	Architecture en couches	13
3.1	Génération de questions par IA avec repli Gemini vers Grok	18
3.2	Correction hybride d'un quiz et réconciliation d'état	20
3.3	Progression séquentielle par UF et cascade vers le certificat	21
3.4	Contrôle d'accès à granularité fine (exemple de la messagerie)	23
4.1	Architecture de déploiement	33

Liste des tableaux

1.1	Répartition indicative des grandes phases du projet	6
2.1	Rôles applicatifs et responsabilités	7
2.2	Stack technique et justification	16
3.1	Synthèse des points forts techniques et de leur valeur ajoutée	27
4.1	Classes de test backend (backend/src/test/java/ma/iatacademy/api/) .	30
4.2	Tests frontend	31

Introduction générale

La transformation numérique des organismes de formation professionnelle ne se limite plus, aujourd'hui, à la simple mise en ligne de supports de cours. Elle touche l'ensemble du cycle de vie de l'apprenant : son inscription, le suivi minutieux de sa progression, l'évaluation de ses compétences par des modalités variées, l'accompagnement individualisé jusqu'à l'obtention d'un diplôme, et jusqu'à la valorisation de ce diplôme sur le marché de l'emploi. C'est dans ce contexte qu'a été pensé et développé IAT Academy, une plateforme e-learning destinée à un cycle professionnel de deux années dans les métiers de l'aviation, de l'accueil et du tourisme — des filières où l'employabilité concrète des diplômés constitue l'indicateur de réussite ultime, bien plus qu'un simple taux de réussite académique.

Avant ce projet, la gestion pédagogique d'IAT Academy s'appuyait sur des outils et des supports dispersés, sans espace numérique centralisé permettant de suivre en temps réel la progression d'un apprenant à travers les différentes unités de formation qui composent son cursus, ni de garantir que les étapes-clés du parcours — en particulier celles qui exigent un jugement humain, comme la validation du stage en entreprise — soient réellement respectées avant qu'un module supérieur ne devienne accessible. La problématique centrale de ce projet peut donc se formuler ainsi : *comment concevoir et développer une plateforme qui structure, sécurise et automatise l'intégralité du parcours pédagogique d'un apprenant, de la candidature à la délivrance d'un certificat vérifiable, tout en respectant des règles métier qui ne se réduisent jamais à un simple calcul automatique de score ?*

Pour répondre à cette problématique, ce stage de fin d'études a consisté à concevoir et implémenter, de bout en bout, une application composée d'un backend Spring Boot exposant une API REST sécurisée et d'un frontend Next.js consommant cette API. Le travail réalisé couvre l'ensemble des étapes d'un projet logiciel réel : l'analyse des besoins et la modélisation du domaine métier, la conception d'une architecture en couches capable de supporter des règles de progression complexes, la mise en œuvre de mécanismes qui dépassent largement le simple CRUD — notamment une intégration à des services d'intelligence artificielle générative pour assister la création de contenu pédagogique — ainsi que la validation de l'ensemble par une suite de tests automatisés et une démarche de déploiement conteneurisé.

Ce rapport est organisé en quatre chapitres. Le premier chapitre présente le contexte général du projet : l'organisme d'accueil, la problématique métier identifiée et les objectifs poursuivis. Le deuxième chapitre détaille l'analyse des besoins et les choix de conception qui en découlent

— acteurs, cas d'utilisation, modèle de données et architecture logicielle. Le troisième chapitre, cœur technique de ce rapport, présente l'implémentation réalisée en mettant l'accent sur les mécanismes qui constituent la véritable valeur ajoutée du projet, classés par ordre d'impact et systématiquement rattachés au code source qui les concrétise. Le quatrième et dernier chapitre décrit la démarche de validation par les tests et l'architecture de déploiement retenue. Une conclusion générale revient enfin sur les apports de ce travail et ouvre sur ses perspectives d'évolution.

Chapitre 1

Contexte général du projet

1.1 Organisme d'accueil

IAT Academy est une académie de formation professionnelle spécialisée dans les métiers de l'aviation, de l'accueil et du tourisme, préparant en particulier aux filières Hôtesse de l'air et Steward, Agent d'escale, Accompagnateur touristique, et Réception hôtelière. La formation se déroule sur un cycle professionnel de deux années, structuré autour d'un équilibre volontairement assumé entre théorie et pratique : cours en présentiel dispensés par des formateurs issus directement du secteur (compagnies aériennes, groupes hôteliers, agences de voyage), stage en milieu réel intégré au cursus plutôt que relégué à une simple option de fin de parcours, et, désormais, un espace d'apprentissage numérique prolongeant la salle de classe au-delà des heures de cours.

Cette double identité — académie de formation aux métiers du transport aérien et du tourisme, mais aussi organisme dont l'indicateur de réussite premier reste l'insertion professionnelle effective de ses diplômés — irrigue l'ensemble des choix fonctionnels du projet. Elle explique pourquoi le référentiel de compétences se traduit directement dans l'organisation du contenu pédagogique (unités de formation alignées sur les compétences métier attendues par les employeurs du secteur), pourquoi le stage occupe une place aussi centrale dans le modèle de progression (voir chapitre 2), et pourquoi la plateforme intègre jusqu'à un mécanisme de certification vérifiable publiquement et de partage professionnel sur les réseaux sociaux (voir chapitre 3) — un choix qui, sur un plan strictement académique, pourrait sembler périphérique, mais qui prend tout son sens dès lors que l'on considère la mission réelle de l'académie : préparer ses apprenants non pas seulement à réussir des examens, mais à être recrutés. L'identité visuelle de la plateforme reprend d'ailleurs ce parti pris métier jusque dans son vocabulaire d'interface, construit autour de la métaphore de la carte d'embarquement — un choix esthétique qui ancre délibérément l'expérience numérique dans l'univers professionnel visé par les apprenants.

1.2 Contexte et problématique

Avant ce projet, le suivi pédagogique d'IAT Academy reposait sur des supports et des outils non centralisés : le contenu de cours, l'évaluation des apprenants et le suivi de leur progression n'étaient réunis dans aucun espace numérique unique. Cette situation, courante dans de nombreux établissements de formation professionnelle de taille intermédiaire, posait en pratique plusieurs difficultés concrètes que ce projet s'est attaché à résoudre une à une.

La première difficulté tenait à l'absence de toute vue d'ensemble, en temps réel, de la progression d'un apprenant à travers les deux années de son cursus. Sans outil centralisé, savoir précisément où en est un apprenant donné — quels modules il a complétés, quelles évaluations lui restent à passer, si son stage a été validé — exigeait de recouper manuellement plusieurs sources d'information, un exercice difficilement soutenable à l'échelle de plusieurs promotions. La seconde difficulté concernait l'évaluation elle-même : les quiz, devoirs et réponses libres des apprenants n'étaient ni centralisés, ni tracés de façon homogène, rendant la correction et le suivi des notes dépendants de pratiques individuelles plutôt que d'un processus outillé et cohérent.

La troisième difficulté, sans doute la plus structurante pour la conception technique du projet, était l'absence de tout mécanisme fiable de validation d'étape : rien, dans l'organisation existante, n'empêchait techniquement un apprenant d'accéder à du contenu avancé sans avoir réellement validé les prérequis correspondants — une lacune d'autant plus problématique que certaines étapes du cursus d'IAT Academy, comme la validation du stage en entreprise ou de la soutenance finale, ne peuvent pas se réduire à un critère automatique et exigent le jugement explicite du Directeur pédagogique. Enfin, la délivrance et la vérification des certificats de réussite reposaient sur un processus entièrement manuel, sans preuve numérique qu'un tiers extérieur — un employeur, un partenaire du secteur — puisse vérifier de façon autonome.

Cette analyse conduit à formuler la problématique centrale de ce projet de la manière suivante : *comment structurer, sécuriser et automatiser le parcours pédagogique complet d'un apprenant — de l'inscription à la délivrance du certificat — tout en respectant des règles métier qui ne se réduisent jamais à de simples critères automatiques, puisque certaines étapes-clés exigent explicitement l'implication du Directeur pédagogique ?* C'est cette tension entre automatisation et jugement humain, plus que la seule numérisation d'un contenu de cours, qui a guidé l'ensemble des choix de conception présentés au chapitre 2.

1.3 Objectifs du projet

Pour répondre à cette problématique, six objectifs concrets ont structuré le travail réalisé durant ce stage.

Le premier objectif consistait à concevoir un modèle de données représentant fidèlement la structure pédagogique réelle du cursus — Formation, Année, Unité de Formation, Module, Leçon, puis Bloc de contenu — sans pour autant complexifier inutilement le schéma relationnel

sous-jacent ; ce choix de modélisation, en apparence anodin, s’est révélé être l’une des décisions de conception les plus structurantes du projet, comme le détaille la section 2.2 du chapitre suivant. Le deuxième objectif visait à mettre en place un système d’évaluation complet, capable de gérer plusieurs types de quiz selon leur portée — section, module, fin d’unité de formation, fin d’année — ainsi que plusieurs types de questions, avec à la fois une correction automatique pour les questions à choix et une correction manuelle pour les réponses libres, ces deux modes devant in fine converger vers un seul et même score final cohérent.

Le troisième objectif était de garantir un déblocage séquentiel du contenu pédagogique fondé sur des règles vérifiées côté serveur, et non pas seulement suggérées côté interface — une exigence de sécurité autant que de fiabilité pédagogique. Le quatrième objectif consistait à fournir à l’équipe pédagogique d’IAT Academy un espace d’administration complet, lui permettant de produire le contenu de cours, de corriger les évaluations, de suivre individuellement chaque apprenant et de réaliser les validations manuelles qui, comme évoqué plus haut, ne peuvent être déléguées à un algorithme. Le cinquième objectif visait à automatiser la délivrance de certificats numériquement vérifiables, tout en outillant le suivi du stage jusqu’à sa signature par un tuteur externe à l’académie. Enfin, le sixième objectif portait sur la sécurisation transversale de l’ensemble de la plateforme — authentification, séparation stricte des rôles, protection contre les abus des fonctionnalités exposées à des services tiers — et sur son déploiement de façon reproductible, condition nécessaire à une exploitation réelle au-delà du cadre du stage.

1.4 Méthodologie de travail et planning

Le développement du projet a suivi une démarche itérative, par fonctionnalité plutôt que par couche technique : chaque bloc fonctionnel — authentification, catalogue de cours, système de quiz, gestion du stage, certification, puis les fonctionnalités transverses de communication et de sécurité — a été conçu, implémenté et testé de bout en bout (du modèle de données jusqu’à l’interface) avant de passer au suivant, plutôt que de développer séparément l’intégralité du backend puis l’intégralité du frontend. Cette approche a permis de valider très tôt, sur des cas concrets, que le modèle de données retenu tenait effectivement la charge des règles métier réelles — en particulier la logique de progression par unité de formation — avant d’investir du temps de développement sur les fonctionnalités qui en dépendent.

Des points d’avancement réguliers avec l’encadrant professionnel au sein d’IAT Academy ont permis de confronter les choix techniques aux contraintes réelles de l’académie, notamment sur les points où une règle métier n’était pas immédiatement évidente à traduire en logique applicative — la distinction entre les unités de formation exigeant une validation manuelle du Directeur et celles n’en exigeant pas en est l’exemple le plus révélateur (voir section 3.2.3).

Le temps du stage s’est réparti, dans ses grandes masses, entre une phase initiale de cadrage et d’analyse des besoins auprès de l’équipe pédagogique, une phase de conception du modèle de données et de l’architecture — la plus déterminante pour la suite, dans la mesure où une erreur

de modélisation à ce stade se serait répercutée sur l'ensemble du développement —, une phase de développement à proprement parler, volontairement la plus longue, menée fonctionnalité par fonctionnalité comme décrit ci-dessus, et enfin une phase de consolidation regroupant les tests, la correction des anomalies détectées et la rédaction du présent rapport. Le tableau 1.1 synthétise cette répartition ; il peut être complété par un diagramme de Gantt détaillé si le format attendu par l'établissement l'exige.

TABLE 1.1 – Répartition indicative des grandes phases du projet

Phase	Contenu principal
Cadrage et analyse des besoins	Immersion dans le fonctionnement d'IAT Academy, recueil des règles métier, définition du périmètre.
Conception	Modélisation des entités, choix d'architecture, définition des mécanismes de sécurité.
Développement	Implémentation backend et frontend, fonctionnalité par fonctionnalité.
Consolidation	Rédaction et exécution des tests, correction des anomalies, rédaction du rapport.

1.5 Conclusion

Ce chapitre a présenté le contexte métier d'IAT Academy, la problématique qui a motivé ce projet — automatiser un parcours pédagogique complexe sans sacrifier les validations qui exigent un jugement humain — ainsi que les objectifs et la méthodologie qui en découlent. Le chapitre 2 détaille maintenant l'analyse fonctionnelle et les choix de conception qui répondent concrètement à ces objectifs.

Chapitre 2

Analyse et conception

2.1 Acteurs et cas d'utilisation

Le système distingue cinq rôles internes, organisés en une hiérarchie de sécurité (`ROLE_SUPER_ADMIN` > `ROLE_ADMIN`), à laquelle s'ajoutent deux acteurs externes ne possédant pas de compte : le tuteur de stage (accès par lien à usage unique) et le visiteur public (formulaire de contact, info-lettre, vérification de certificat).

TABLE 2.1 – Rôles applicatifs et responsabilités

Rôle	Responsabilités
<code>SUPER_ADMIN</code>	Paramètres système globaux, en plus de tous les droits Admin.
<code>ADMIN</code> (Directeur)	Gestion pédagogique complète, validation des UF sensibles, gestion des groupes et des campagnes email, en plus de tous les droits Formateur.
<code>FORMATEUR</code>	Studio de cours, gestion des quiz, correction manuelle, messagerie, bourse à l'emploi.
<code>ETUDIANT</code>	Espace apprenant : contenu, quiz, stage, certificat, messagerie.
<code>SUPPORT</code>	Accès limité, essentiellement en lecture.

La Figure 2.1 synthétise l'ensemble des cas d'utilisation identifiés à partir des 29 contrôleurs REST exposés par le backend.

attributs et cardinalités : structure pédagogique et groupes (Figure 2.3), évaluation (Figure 2.4), et stage et certification (Figure 2.5). Les domaines restants (communication, marketing/emploi, médias, suivi système) suivent les mêmes conventions de modélisation et sont détaillés au besoin dans le code source (domain/entity/).

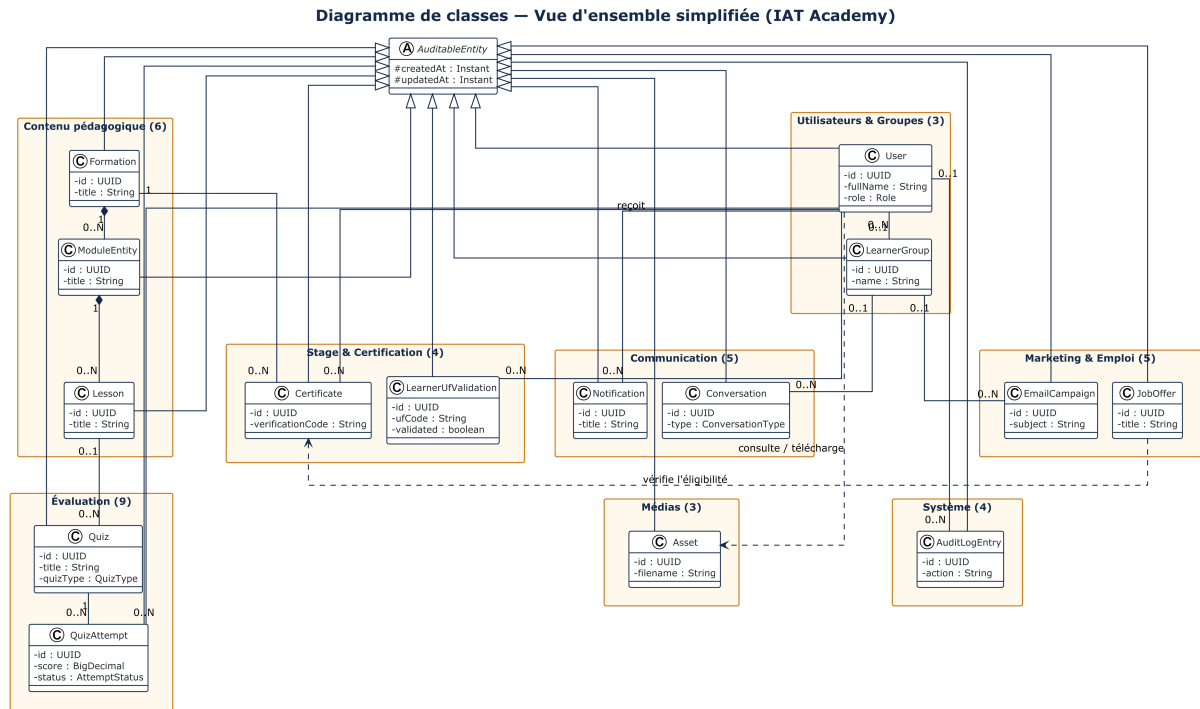


FIGURE 2.2 – Diagramme de classes — vue d'ensemble simplifiée de l'application

Diagramme de classes — Structure pédagogique et groupes

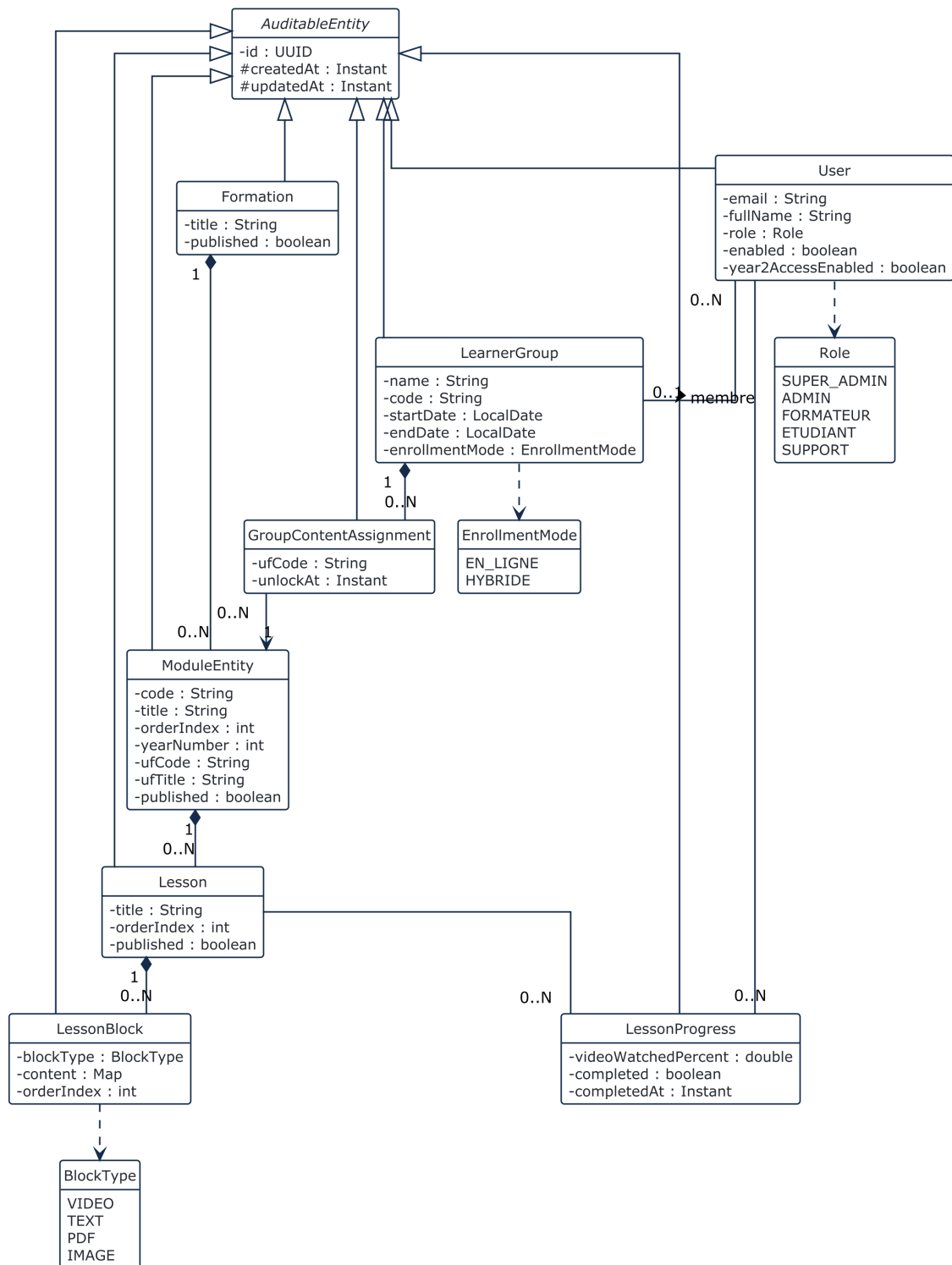


FIGURE 2.3 – Diagramme de classes — structure pédagogique et groupes

Diagramme de classes — Évaluation (quiz, devoirs, notation)

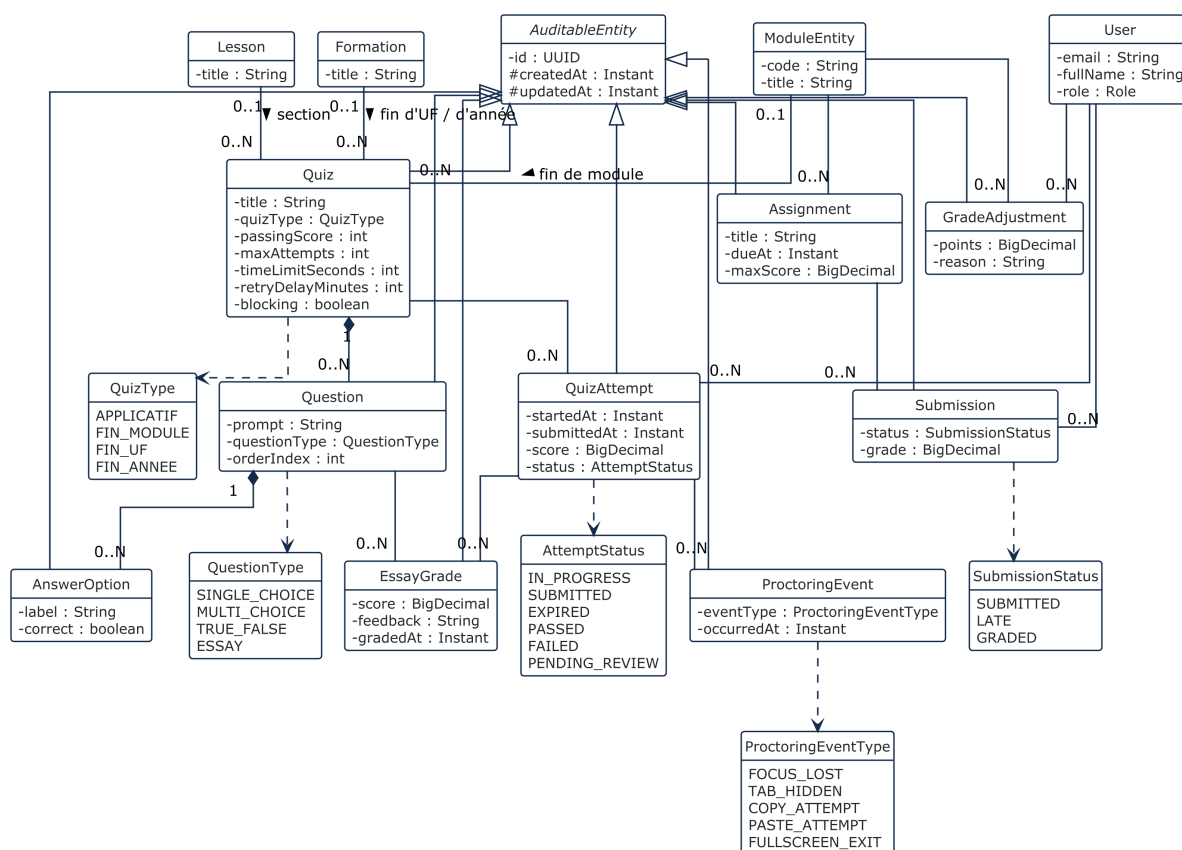


FIGURE 2.4 – Diagramme de classes — évaluation

Diagramme de classes — Stage et certification

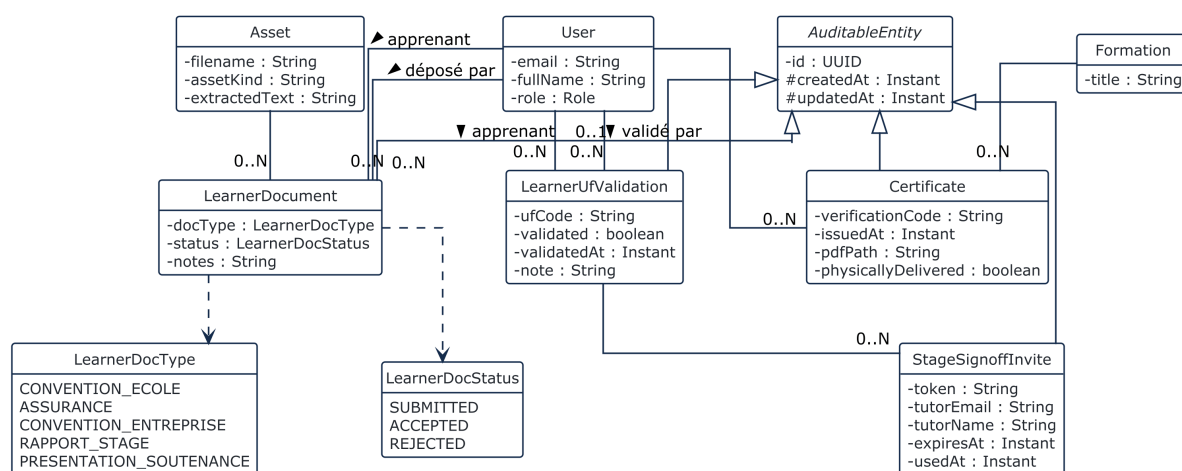


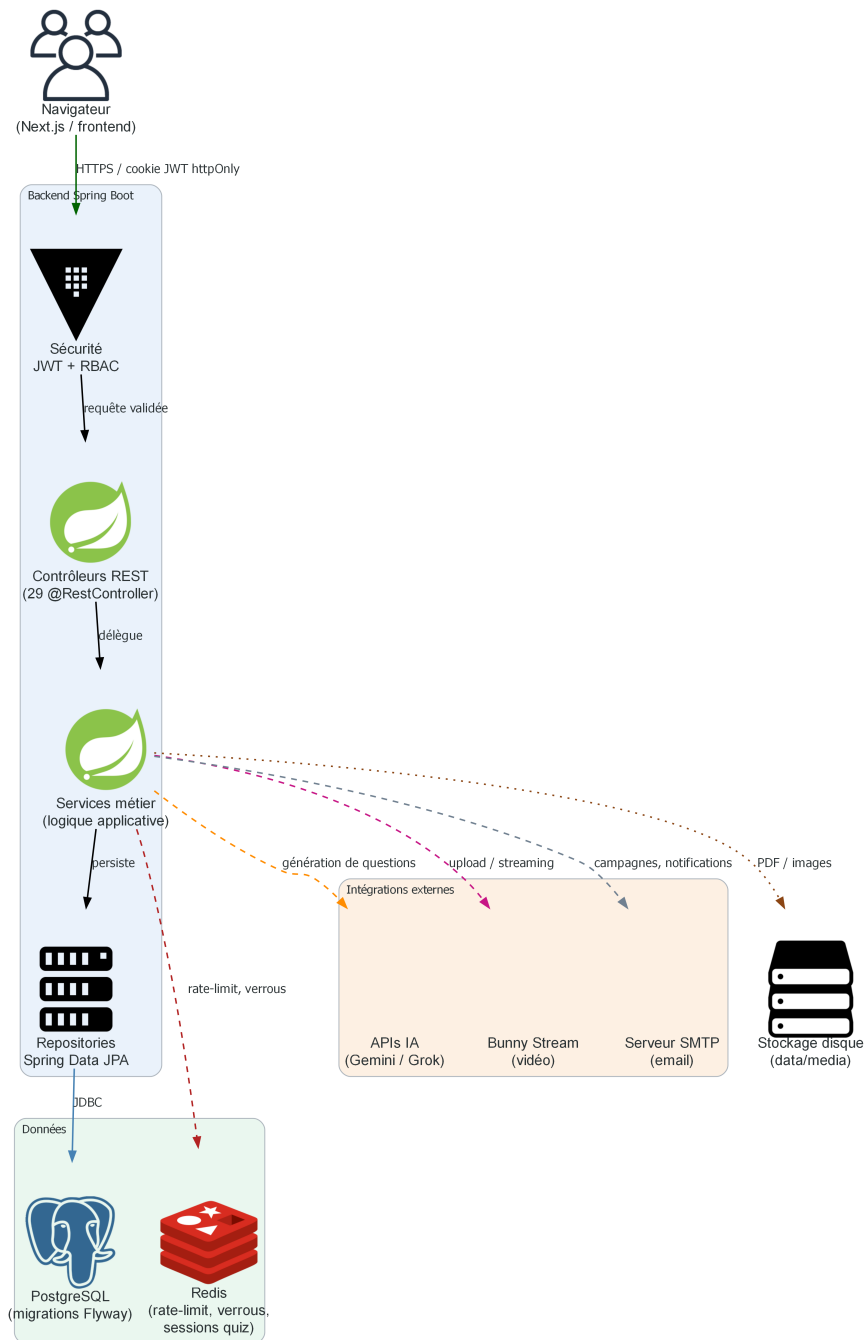
FIGURE 2.5 – Diagramme de classes — stage et certification

2.2.1 Un choix de modélisation délibéré : l'UF comme attribut

Un choix de conception mérite d'être souligné : l'Unité de Formation (UF) et l'année du cursus ne sont *pas* modélisées comme des entités séparées avec leur propre table et clé primaire. Ce sont des attributs (`ufCode`, `ufTitle`, `yearNumber`) portés directement par `ModuleEntity`. Ce choix évite une jointure supplémentaire pour une notion qui, dans les règles métier réelles, sert uniquement de *clé de regroupement* (voir §3.2.3) — il garde le schéma relationnel simple sans sacrifier la logique de progression par UF, qui est recalculée dynamiquement à partir de ces attributs plutôt que stockée redondamment.

2.3 Architecture logicielle

L'application suit une architecture en couches classique (Figure 2.6) : `Controller` → `Service` → `Repository` → `Entity`, adossée à PostgreSQL pour la persistance et Redis pour les besoins transverses (limitation de débit, verrous, état éphémère des tentatives de quiz).



Architecture en couches - IAT Academy

FIGURE 2.6 – Architecture en couches

La règle de conception appliquée de façon constante dans le code est la suivante : un contrôleur ne fait que du routage HTTP, de la validation `@PreAuthorize` et de la traduction requête/réponse ; toute la logique métier — y compris les vérifications de propriété de ressource détaillées au chapitre 3 — vit dans la couche service, jamais dans le contrôleur.

2.4 Sécurité

La sécurité de la plateforme ne pouvait pas être traitée comme une simple case à cocher en fin de développement : dès lors qu’une partie du contenu — réponses de quiz, dossiers de stage, échanges de messagerie — touche directement à la vie privée et à l’évaluation d’un apprenant mineur ou jeune majeur, l’authentification et le contrôle d’accès devaient être pensés comme des contraintes de conception à part entière, et non comme une couche ajoutée après coup. Cette section détaille les deux piliers sur lesquels repose cette sécurité : l’authentification, qui répond à la question *qui êtes-vous ?*, et l’autorisation, qui répond à la question, en apparence proche mais en réalité bien plus difficile à garantir correctement, *avez-vous le droit d’agir sur cette ressource précise ?*

2.4.1 Authentification

L’authentification de la plateforme repose sur un jeton JWT (signé en HS256) transporté par un cookie `httpOnly` — c’est-à-dire un cookie que le JavaScript exécuté dans le navigateur ne peut ni lire ni manipuler, ce qui ferme la voie la plus commune d’exfiltration de jeton en cas de faille de type *cross-site scripting* (XSS) sur le frontend. Le déroulement complet, depuis la soumission du formulaire de connexion jusqu’à l’établissement du contexte de sécurité sur chaque requête suivante, se décompose en quatre étapes. Premièrement, `AuthService.login()` authentifie les identifiants soumis par l’intermédiaire de l’`AuthenticationManager` de Spring Security, qui compare le mot de passe fourni au hachage BCrypt stocké en base — jamais le mot de passe en clair. Deuxièmement, si l’utilisateur a activé la double authentification par application TOTP (Google Authenticator ou équivalent), la réponse renvoyée à ce stade n’est pas encore le cookie de session final, mais un jeton temporaire valable cinq minutes, qui exige un second appel explicite à l’endpoint `/verify-2fa` avant que la connexion ne soit réellement effective — une étape supplémentaire délibérément conçue pour qu’un mot de passe compromis seul ne suffise jamais à usurper un compte protégé par 2FA. Troisièmement, une fois l’authentification (simple ou renforcée par 2FA) confirmée, `JwtService` construit le jeton définitif, dont le sujet est l’identifiant de l’utilisateur et dont les revendications embarquent son adresse email et son rôle applicatif. Quatrièmement, à chaque requête suivante, `JwtAuthenticationFilter` — un filtre Spring exécuté avant le filtre d’authentification standard du framework — extrait ce jeton du cookie, le valide, puis peuple le contexte de sécurité de la requête courante avant que celle-ci n’atteigne le contrôleur ciblé.

Ce choix architectural a une conséquence directe sur la politique de session retenue : le backend fonctionne en mode STATELESS, c’est-à-dire qu’aucune session n’est conservée en mémoire côté serveur. L’intégralité de l’état d’authentification est reconstruite, à chaque requête, à partir du seul contenu du jeton — un choix cohérent avec une architecture où le frontend Next.js est physiquement découplé du backend Spring Boot, et qui simplifie d’autant la mise à l’échelle horizontale de l’API si elle devait un jour être répliquée sur plusieurs instances, puisqu’aucune

session partagée n’a besoin d’être synchronisée entre elles.

2.4.2 Autorisation

Savoir *qui* fait la requête ne suffit pas à savoir si cette requête doit être acceptée : le contrôle d’accès de la plateforme combine, de façon volontairement distincte, deux niveaux de vérification qui ne répondent pas à la même question. Le premier niveau est un **RBAC déclaratif**, appliqué au moyen de l’annotation `@PreAuthorize` posée sur chaque endpoint sensible, et résolu contre la hiérarchie de rôles `ROLE_SUPER_ADMIN > ROLE_ADMIN` présentée au chapitre 3. Ce niveau répond correctement à la question « cet utilisateur a-t-il le *type* de compte requis pour accéder à cette fonctionnalité ? », mais il est structurellement incapable de répondre à une seconde question, tout aussi critique dès lors qu’une ressource appartient nommément à un utilisateur précis : « cet utilisateur a-t-il le droit d’accéder à *cette instance précise* de la ressource, et pas seulement à des ressources de ce type en général ? ». C’est pour répondre à cette seconde question qu’un second niveau, un **contrôle de propriété** implémenté au niveau de la couche service, vient compléter le RBAC pour les ressources intrinsèquement personnelles — une conversation de messagerie, une notification, un dossier de stage — comme le détaille et l’illustre concrètement le §3.2.4 du chapitre suivant.

2.5 Choix technologiques

Chaque brique technologique retenue pour ce projet répond à une contrainte identifiée dans les sections précédentes, plutôt qu’à un simple effet de mode : le Tableau 2.2 met en regard chaque choix et la justification qui le sous-tend, en particulier pour les décisions les moins évidentes de prime abord — pourquoi Redis en plus d’une base relationnelle déjà présente, ou pourquoi un rendu serveur côté frontend pour une seule page de la plateforme.

TABLE 2.2 – Stack technique et justification

Technologie	Justification
Spring Boot 3 / Java 21	Écosystème mature, sécurité (Spring Security) et accès aux données (Spring Data JPA) intégrés, adapté à une application métier avec des règles de progression complexes.
PostgreSQL + Flyway	Base relationnelle fiable pour un modèle fortement structuré (formations/modules/leçons); migrations versionnées reproductibles entre environnements.
Redis	Structures de données en mémoire adaptées aux besoins transverses à courte durée de vie (compteurs de débit, verrous d'édition, état de tentative de quiz) sans alourdir PostgreSQL de lectures/écritures fréquentes et volatiles.
JWT + cookie httpOnly	Authentification sans état, compatible avec un frontend découplé (Next.js), sans exposer le jeton au JavaScript (protection XSS).
Next.js 15 (frontend)	Rendu serveur pour les pages publiques exigeant des métadonnées SEO/Open Graph (ex. vérification de certificat, voir §3.2.6).

2.6 Conclusion

Ce chapitre a présenté les cas d'utilisation, le modèle de données, les choix d'architecture et de sécurité qui structurent IAT Academy. Le chapitre 3 détaille maintenant comment ces choix se traduisent dans l'implémentation, en particulier sur les mécanismes qui dépassent le CRUD simple.

Chapitre 3

Réalisation

3.1 Vue d'ensemble des modules implémentés

Le backend expose 29 contrôleurs REST couvrant l'ensemble du cycle de vie pédagogique : authentification (avec 2FA optionnelle), catalogue de cours et éditeur de contenu par blocs, système de quiz à quatre portées (section, module, UF, année), gestion de stage avec signature externe, certification, messagerie, notifications, campagnes email, bourse à l'emploi, recherche globale et outils d'administration (utilisateurs, groupes, journal d'audit, paramètres système). Plutôt que de décrire exhaustivement chaque endpoint CRUD — déjà couvert au chapitre 2 — ce chapitre se concentre sur les mécanismes qui *dépassent* le CRUD simple et qui constituent la valeur ajoutée technique réelle du projet.

3.2 Points forts techniques et valeur ajoutée

Les onze points suivants sont classés par ordre d'impact décroissant : du plus démonstratif d'une compétence technique ou d'une valeur ajoutée utilisateur, au plus secondaire. Chaque affirmation renvoie à un fichier et, quand c'est utile, à une ligne précise du code source.

3.2.1 Génération de questions de quiz par intelligence artificielle, avec repli entre fournisseurs

Contexte du problème. Rédiger manuellement des questions de quiz pour chaque leçon est chronophage pour l'équipe pédagogique. Un service d'IA générative peut accélérer ce travail, mais dépendre d'un seul fournisseur externe (quota, panne, latence) rendrait la fonctionnalité peu fiable en production.

Solution apportée. `QuestionGenerationService` (`service/QuestionGenerationService.java`) interroge d'abord l'API Gemini (`callGemini`, ligne 120); en cas d'échec — réseau, réponse HTTP non-2xx, JSON invalide, ou aucune question extraite — il bascule automatiquement vers

l'API Grok (`callGrok`, ligne 140), avant d'échouer explicitement si les deux fournisseurs sont indisponibles. Les deux méthodes sont volontairement `package-private` (et non `private`) pour être simulées (`Mockito.spy()`) dans `QuestionGenerationServiceTest` sans appel réseau réel. Le résultat brut est nettoyé (extraction hors des blocs `""json`), parsé, puis chaque question générée passe par *exactement* la même validation (`addQuestion`) qu'une question créée manuellement — une question invalide dans le lot est rapportée comme erreur individuelle sans annuler les autres. L'appel est en outre protégé par `RateLimitService.checkAiGenerationAllowed` (20 requêtes / 5 minutes par utilisateur) pour contenir le coût d'un appel à une API payante.

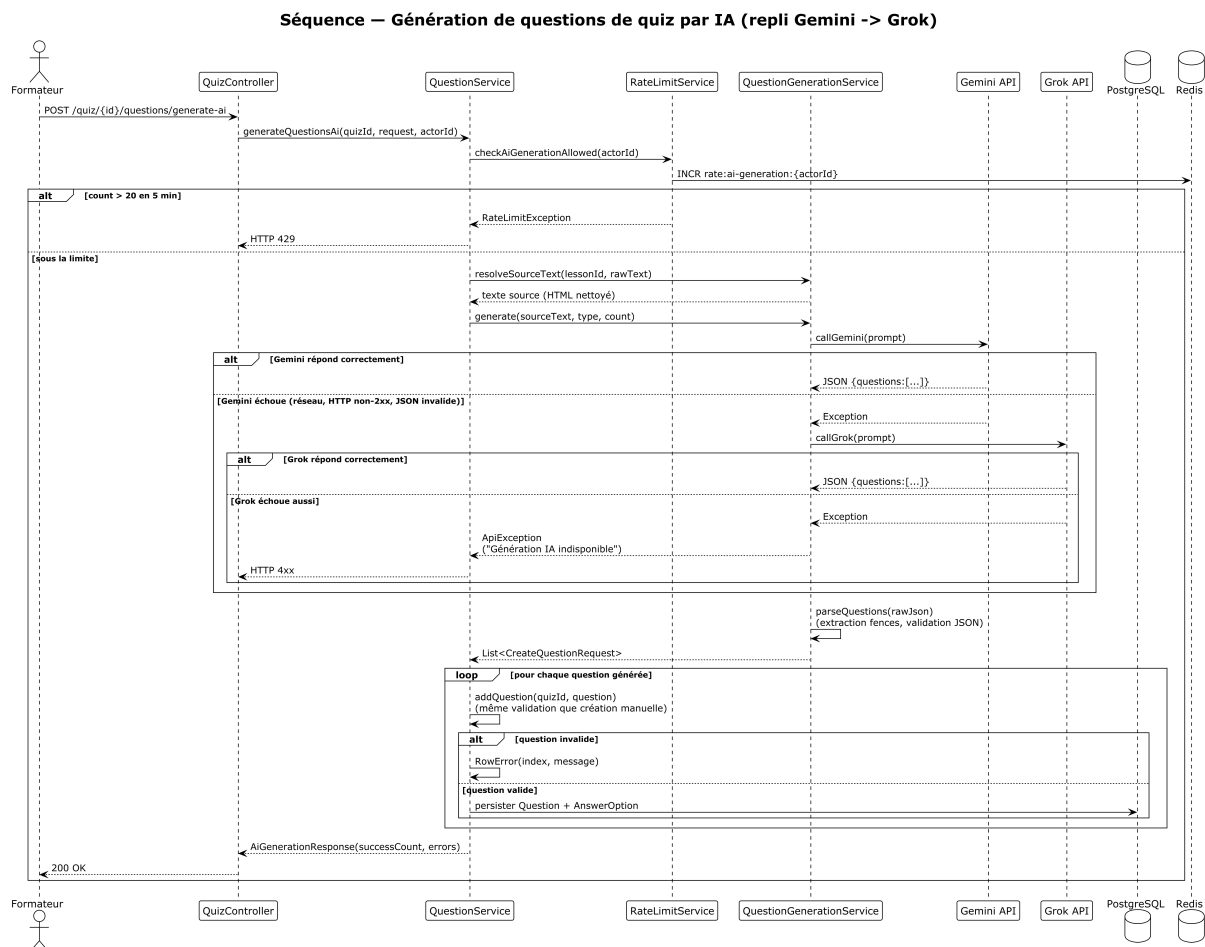


FIGURE 3.1 – Génération de questions par IA avec repli Gemini vers Grok

Bénéfice concret. Une simple intégration à un seul fournisseur aurait suffi pour une démonstration, mais pas pour un usage réel en production, où une panne ou un changement de quota chez Gemini aurait purement et simplement désactivé la fonctionnalité. Le mécanisme de repli et la réutilisation de la validation existante garantissent que le contenu généré par IA respecte les mêmes invariants qu'un contenu créé à la main, sans code de persistance dupliqué.

3.2.2 Correction hybride des quiz : automatique et manuelle, avec réconciliation d'état

Contexte du problème. Un même quiz peut mélanger des questions à choix (auto-corrigeables) et des questions à réponse libre (nécessitant un jugement humain). Une solution naïve traiterait chaque soumission comme « terminée » dès l'envoi, ce qui est incorrect si une partie de la note dépend encore d'une correction à venir.

Solution apportée. `QuizAttemptService.submit()` (`service/QuizAttemptService.java`) calcule le score des questions auto-corrigeables par égalité stricte d'ensembles d'identifiants d'options sélectionnées — ce qui signifie qu'un choix multiple ne donne *aucun* crédit partiel : une réponse presque correcte compte comme fausse. Si le quiz contient au moins une question `ESSAY`, la tentative passe au statut `PENDING_REVIEW` plutôt que `PASSED/FAILED`. `QuizGradingService.grade` persiste chaque note manuelle, puis `finalizeIfFullyGraded()` vérifie si *toutes* les questions à réponse libre de la tentative ont désormais une note — ce n'est qu'à ce moment que le score final est recalculé et que la tentative bascule vers son statut définitif, en réutilisant `applyPassEffects()` — la même méthode qui gère badges, notifications et éligibilité au certificat pour une soumission entièrement automatique. Une autre difficulté résolue : le démarrage concurrent d'une tentative est sérialisé par un verrou Postgres (`pg_advisory_xact_lock`) pour empêcher qu'un double clic ne crée deux tentatives `IN_PROGRESS` simultanées — comportement vérifié par un script de test de charge (`loadtest/quiz-start-concurrency.js`).

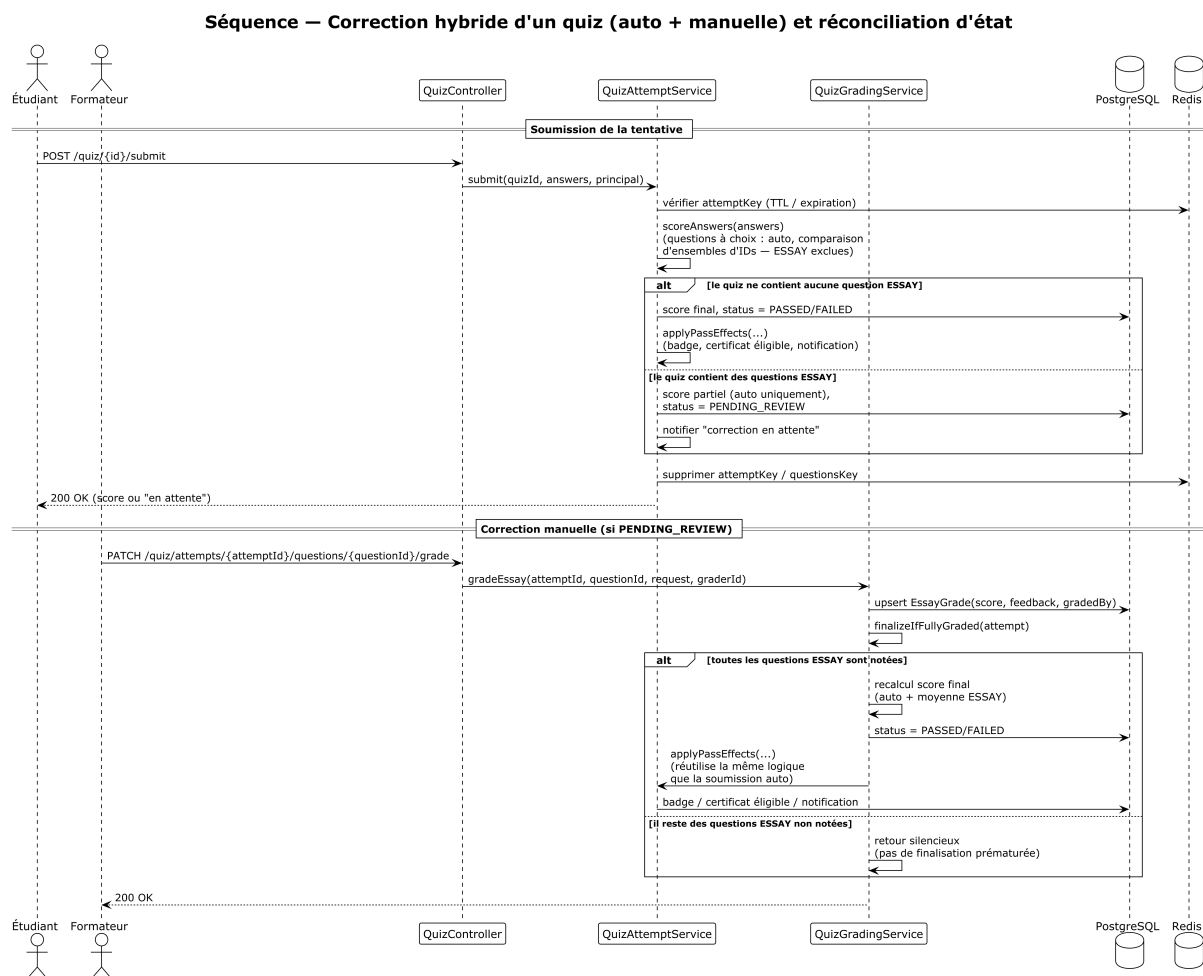


FIGURE 3.2 – Correction hybride d'un quiz et réconciliation d'état

Bénéfice concret. Sans cette réconciliation explicite, un formateur pourrait involontairement déclencher une notification « quiz réussi » ou l'émission d'un certificat avant qu'une réponse libre ait été relue — une erreur silencieuse difficile à détecter en production. La réutilisation de `applyPassEffects()` garantit aussi qu'un apprenant reçoit exactement le même traitement (badge, certificat) qu'il ait été corrigé automatiquement ou manuellement.

3.2.3 Progression pédagogique séquentielle par UF, avec cascade vers l'éligibilité au certificat

Contexte du problème. Le cursus impose un ordre : un apprenant ne doit pas pouvoir accéder au contenu d'une UF avant d'avoir complété la précédente, ni obtenir un certificat sans avoir réellement terminé l'intégralité du parcours — y compris les étapes qui exigent un jugement humain (validation du stage par le Directeur).

Solution apportée. `ProgressionService.isUfFullyDone()` (voir aussi la note du diagramme de classes, Figure 2.3) combine trois conditions : le contenu de tous les modules de l'UF est complété, le quiz de fin d'UF est réussi *si* il existe (sinon la condition est neutre), et

UfValidationService.isValidated() confirme une validation manuelle — mais uniquement pour les deux UF qui l'exigent réellement (UF 5 = stage, UF 11 = soutenance; toutes les autres UF renvoient true sans intervention humaine). Les apprenants rattachés à un groupe (cohorte présentielle/hybride) échappent à cette règle séquentielle automatique : leur accès est piloté explicitement par GroupContentAssignment, avec des dates d'ouverture fixées par le Directeur — deux stratégies de déblocage cohabitant dans le même modèle selon le profil d'apprenant. En bout de chaîne, CertificateService.tryIssueIfEligible() ne délivre un certificat que si *tous* les modules de la formation sont complétés — la cascade UF → module → certificat n'est jamais court-circuitée.

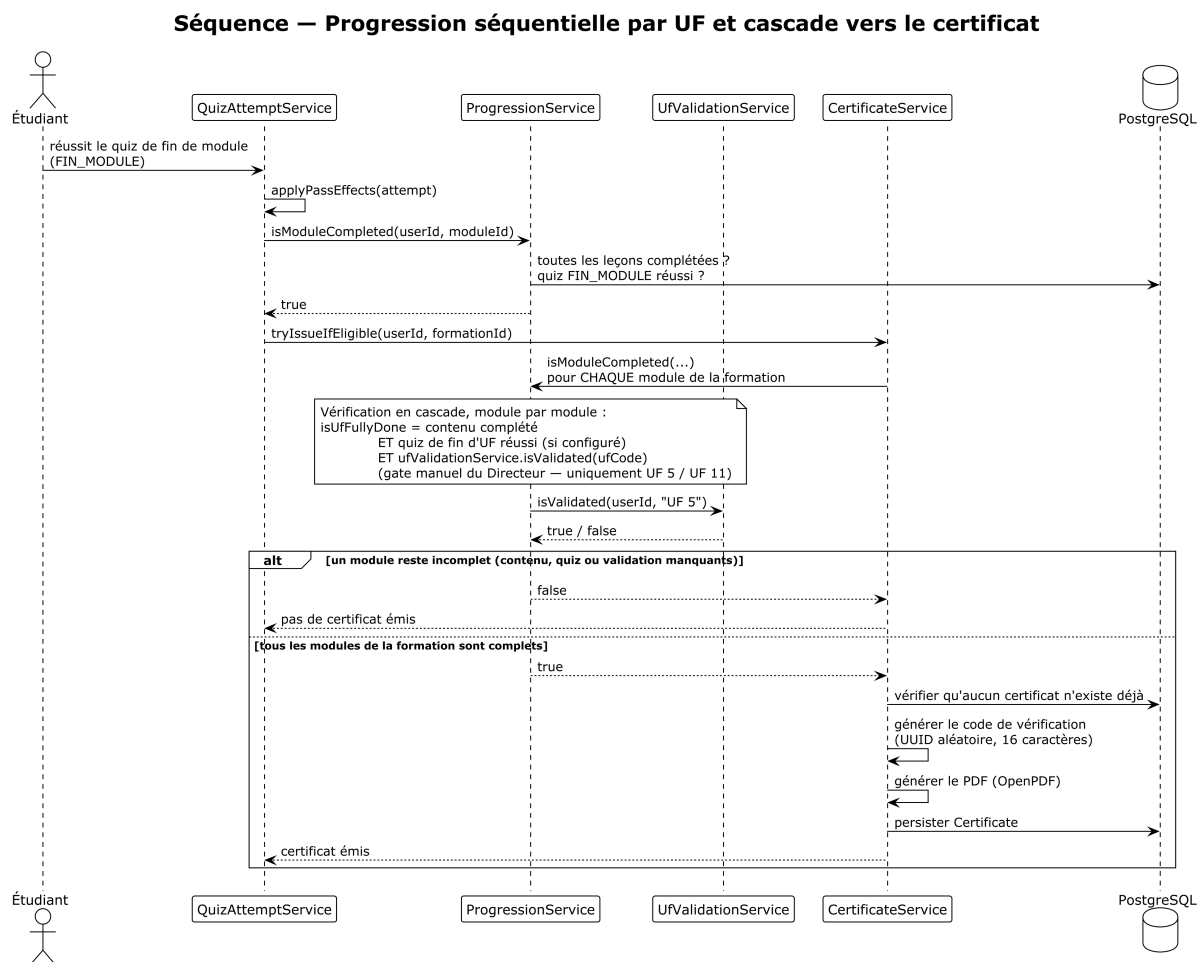


FIGURE 3.3 – Progression séquentielle par UF et cascade vers le certificat

Bénéfice concret. Un modèle plus simple (un simple pourcentage de progression global) n'aurait pas pu représenter la contrainte réelle du Directeur : certaines étapes ne se valident pas automatiquement, quel que soit le score obtenu. La double stratégie individuel/groupe permet en outre au même système de servir aussi bien les apprenants en ligne autonomes que les cohortes encadrées en présentiel.

3.2.4 Contrôle d'accès à granularité fine, au-delà du RBAC

Contexte du problème. Un rôle (@PreAuthorize) suffit à dire *qu'un* apprenant peut lire ses messages, pas *lequel*. Sans vérification supplémentaire, un utilisateur authentifié pourrait accéder aux conversations, notifications ou notes personnelles d'un autre utilisateur simplement en devinant un identifiant dans l'URL.

Solution apportée. Plusieurs services implémentent un contrôle de propriété explicite, systématiquement révérifié à *chaque* opération (pas seulement à l'affichage de la liste) : `MessagingService.assess` distingue les conversations directes (vérification d'une ligne `ConversationParticipant`) des salons de cohorte (vérification du groupe partagé, avec accès de plein droit pour le staff); `NotificationService.markRead()` et `LessonNoteService.requireOwnNote()` comparent directement le propriétaire de la ressource à l'utilisateur authentifié; `StageService` va plus loin avec trois méthodes d'assertion distinctes selon le type de document (document réservé au Directeur, document réservable à l'apprenant concerné uniquement, avec en plus une fenêtre de dates pour l'upload).

Séquence — Contrôle d'accès à granularité fine (au-delà du RBAC)

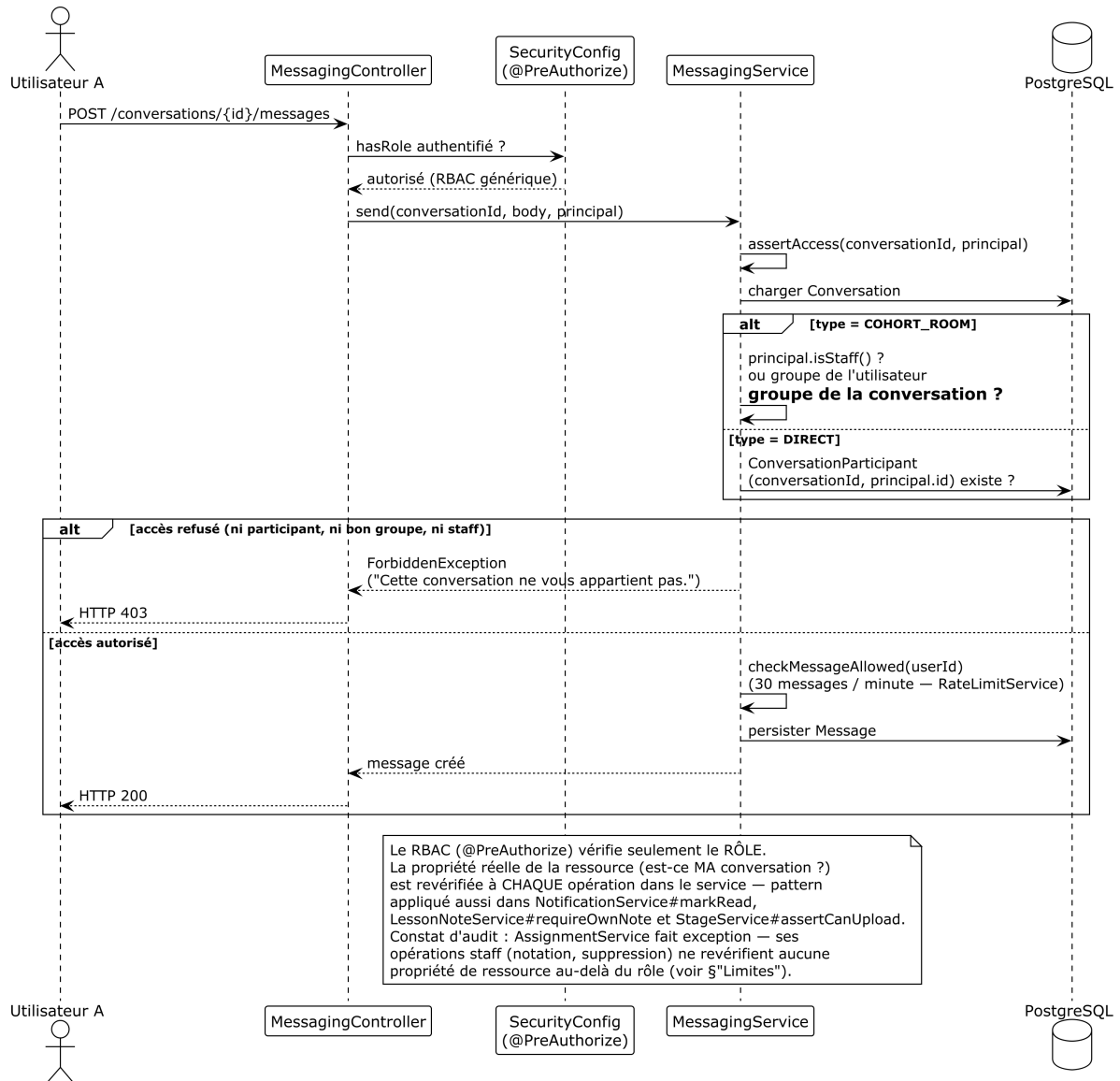


FIGURE 3.4 – Contrôle d'accès à granularité fine (exemple de la messagerie)

Bénéfice concret. Ce pattern — vérification de propriété au niveau service, jamais seulement au niveau contrôleur — ferme une classe entière de vulnérabilités de type IDOR (*Insecure Direct Object Reference*, OWASP Top 10 [1]). Sa mise en œuvre n'est toutefois pas uniforme dans tout le code : voir la limite identifiée au §3.3.

3.2.5 Limitation de débit Redis, appliquée de façon transverse

Contexte du problème. Plusieurs endpoints exposent une surface d'abus possible : tentatives de connexion (force brute), appels à des API IA payantes, envoi de messages (spam), upload de fichiers.

Solution apportée. RateLimitService implémente un compteur à fenêtre fixe avec exac-

tement deux primitives Redis — INCR puis un EXPIRE conditionnel posé uniquement à la création du compteur — exposé via huit méthodes dédiées (`checkLoginAllowed`, `checkAiGenerationAllowed`, `checkMessageAllowed`, etc.), chacune avec son propre seuil et sa propre fenêtre temporelle. Le dépassement lève une `RateLimitException` annotée `@ResponseStatus(TOO_MANY_REQUESTS)`, traduite automatiquement en HTTP 429 sans gestionnaire d'exception dédié.

Bénéfice concret. Un seul mécanisme réutilisable protège à la fois la sécurité (force brute sur la connexion) et le budget d'exploitation (appels IA facturés à l'usage) — sans dépendance supplémentaire, Redis étant déjà présent dans la stack pour d'autres besoins.

3.2.6 Certificats vérifiables publiquement et partage LinkedIn

Contexte du problème. Un certificat PDF classique n'offre aucun moyen à un tiers (recruteur, partenaire) de vérifier son authenticité, et ne valorise pas la réussite de l'apprenant sur les réseaux professionnels.

Solution apportée. `CertificateService` génère un code de vérification aléatoire (UUID tronqué, 16 caractères) exposé par un endpoint public *sans authentification* (`GET /api/certificates/verify/` qui ne renvoie que des informations déjà publiques par nature (nom de l'apprenant, formation, date — jamais d'email ni de PDF téléchargeable). Côté frontend, la page `Next.js /verify/[code]` est un *composant serveur* dont la fonction `generateMetadata` interroge cette API pour produire les balises Open Graph directement dans le HTML initial — condition nécessaire pour que le robot d'indexation de LinkedIn, qui n'exécute pas de JavaScript, affiche un aperçu correct lors du partage.

Bénéfice concret. Cette fonctionnalité transforme un document statique en preuve vérifiable et en outil de visibilité professionnelle pour l'apprenant — une valeur ajoutée directement alignée sur la mission de l'académie (employabilité).

3.2.7 Recherche globale multi-source, avec confidentialité des questions de quiz

Contexte du problème. Une recherche naïve sur les seuls titres de modules/leçons est peu utile. L'étendre au contenu réel (texte, PDF, questions de quiz) introduit un risque : un apprenant pourrait utiliser la recherche pour retrouver directement l'énoncé — voire la réponse — d'une question d'examen.

Solution apportée. `SearchService` interroge cinq sources (titres de modules, titres de leçons, corps de texte des blocs TEXT, texte extrait des blocs PDF, prompts de questions), avec un extrait (*snippet*) calculé autour du premier terme trouvé. `SearchController` détermine `includeQuestions` à partir du rôle du principal authentifié (`isStaff()`) *avant* d'appeler le service : si l'appelant n'est pas membre du staff, le dépôt de questions n'est **jamais interrogé** — la confidentialité n'est pas un simple filtrage d'affichage après coup. L'extraction du texte

des PDF a lieu au moment de l'upload (MediaService, Apache PDFBox), en échec silencieux (best-effort) pour ne jamais bloquer un import de fichier.

Bénéfice concret. La recherche devient un outil pédagogique (retrouver un passage de cours) sans devenir un outil de triche — un compromis qui aurait été manqué par une implémentation qui se contenterait de masquer les résultats côté interface.

3.2.8 Bourse à l'emploi dérivée de la possession d'un certificat

Contexte du problème. L'accès à la bourse à l'emploi doit être réservé aux diplômés — une notion qui pourrait être modélisée par un simple champ booléen `isAlumni` sur `User`, au risque de se désynchroniser de la réalité (diplôme révoqué, erreur manuelle).

Solution apportée. `JobOfferService.listForLearner()` ne stocke aucun champ dédié : le statut de diplômé est *dérivé* de l'existence d'une ligne `Certificate` pour cet utilisateur et cette formation (`certificateRepository.findByIdAndFormationId`), exactement la même requête que celle utilisée pour afficher le certificat lui-même. Un apprenant non diplômé reçoit une erreur 403 explicite (« Réservé aux diplômés »). Le staff, lui, passe par une route `/admin/job-offers` entièrement distincte qui ne traverse jamais cette vérification.

Bénéfice concret. L'état « diplômé » ne peut jamais être incohérent avec la réalité du certificat — il n'existe qu'un seul endroit où cette vérité est stockée (principe *single source of truth*).

3.2.9 Envoi de campagnes email asynchrone à débit maîtrisé

Contexte du problème. Envoyer plusieurs centaines d'emails de façon synchrone bloquerait la requête HTTP de l'administrateur, et un envoi trop rapide risquerait de faire percevoir le serveur SMTP comme source de spam.

Solution apportée. `EmailCampaignService.send()` persiste immédiatement tous les destinataires en statut `PENDING`, bascule la campagne à `SENDING`, puis délègue à `CampaignSenderWorker.sendAs` — une classe séparée spécifiquement parce qu'une méthode `@Async` appelée depuis la même classe contournerait le proxy AOP de Spring et ne serait jamais réellement asynchrone. L'envoi traite les destinataires un par un avec une pause fixe de 200 ms, sur un pool de threads volontairement restreint (2 à 4 threads) pour ne jamais saturer le serveur SMTP. Chaque échec individuel est capturé, journalisé et marqué `FAILED` sans interrompre l'envoi des destinataires suivants.

Bénéfice concret. L'administrateur récupère l'interface immédiatement après avoir lancé l'envoi, et l'état de chaque destinataire (en base, pas dans une file séparée) reste consultable et survit à un redémarrage du serveur — avec une limite assumée décrite au §3.3.

3.2.10 Détection d'inactivité (alerte précoce)

Contexte du problème. Repérer les apprenants décrocheurs permet une intervention pédagogique avant l'échec.

Solution apportée. `EarlyWarningService.findInactiveStudents(days)` calcule, pour chaque apprenant, le nombre de jours écoulés depuis la dernière mise à jour de sa `LessonProgress` — avec repli sur la date de création du compte si aucune progression n'existe encore — et retourne la liste triée par durée d'inactivité décroissante, consultée à la demande depuis le tableau de bord analytique (aucune tâche planifiée, aucun effet de bord automatique).

Bénéfice concret. Un signal simple mais fiable, calculé à partir d'une donnée déjà existante plutôt que d'un nouveau mécanisme de suivi dédié.

3.2.11 Verrou d'édition collaboratif (distinct de la progression séquentielle)

Contexte du problème. Deux membres du staff éditant la même leçon en même temps risqueraient d'écraser mutuellement leurs modifications.

Solution apportée. `LessonLockService` pose un verrou Redis nominatif (`lesson_lock:{lessonId}`, TTL 30 minutes, renouvelable) identifiant qui édite actuellement une leçon ; toute tentative concurrente reçoit une `LessonLockedException` (HTTP 409) nommant la personne qui détient le verrou. *Précision utile pour ce rapport* : ce mécanisme ne doit pas être confondu avec le verrouillage séquentiel du parcours apprenant (§3.2.3, implémenté dans `ProgressionService`) — il s'agit d'une garde d'édition collaborative côté back-office, sans rapport avec la progression pédagogique.

Bénéfice concret. Un conflit d'édition est prévenu plutôt que constaté après coup, avec un message explicite plutôt qu'un écrasement silencieux.

3.2.12 Tableau de synthèse

Le Tableau 3.1 regroupe, dans l'ordre d'impact retenu tout au long de cette section, les onze mécanismes présentés : leur emplacement précis dans le code source, et l'argument de valeur ajoutée qui justifie leur mise en avant — réutilisable tel quel comme support de soutenance.

TABLE 3.1 – Synthèse des points forts techniques et de leur valeur ajoutée

Mécanisme	Localisation	Valeur ajoutée
Génération de questions par IA (repli Gemini/Grok)	QuestionGenerationService	Fonctionnalité IA fiable en production, jamais dépendante d'un seul fournisseur externe.
Correction hybride de quiz (auto + manuelle)	QuizAttemptService, QuizGradingService	Un score final toujours cohérent, jamais figé avant qu'une correction manuelle en attente ne soit terminée.
Progression séquentielle par UF → certificat	ProgressionService, UfValidationService, CertificateService	Respecte les validations humaines obligatoires (stage, soutenance) sans jamais les court-circuiter automatiquement.
Contrôle d'accès à granularité fine	MessagingService, NotificationService, LessonNoteService, StageService	Ferme une classe de vulnérabilités IDOR que le seul RBAC ne peut pas couvrir.
Limitation de débit Redis transverse	RateLimitService	Protège à la fois la sécurité (force brute) et le budget d'exploitation (appels IA payants) avec un seul mécanisme réutilisable.
Certificat vérifiable publiquement + partage LinkedIn	CertificateService, page /verify/[code]	Transforme un diplôme statique en preuve vérifiable et en outil d'employabilité réelle.
Recherche globale avec confidentialité des questions	SearchService, SearchController	Un outil pédagogique qui ne peut jamais devenir un outil de triche.
Bourse à l'emploi dérivée du certificat	JobOfferService	Le statut « diplômé » ne peut jamais se désynchroniser de la réalité (source de vérité unique).
Campagnes email asynchrones à débit maîtrisé	EmailCampaignService, CampaignSenderWorker	L'administrateur n'attend jamais l'envoi complet, sans jamais saturer le serveur SMTP.
Détection d'inactivité	EarlyWarningService	Signal fiable pour l'intervention pédagogique, sans mécanisme de suivi dédié supplémentaire.
Verrou d'édition collaboratif	LessonLockService	Prévient un conflit d'édition entre formateurs plutôt que de le constater après coup.

3.3 Limites identifiées et pistes d'amélioration

Deux limites concrètes, identifiées en analysant le code plutôt que supposées, sont assumées ici plutôt que passées sous silence :

- **Incohérence du contrôle de propriété.** Contrairement aux services décrits au §3.2.4, `AssignmentService.gradeSubmission()` et `AssignmentService.delete()` ne vérifient aucune propriété de ressource au-delà du rôle `@PreAuthorize("hasAnyRole('ADMIN','FORMAT')")`. Un formateur peut ainsi noter ou supprimer un devoir rattaché à un module qu'il n'encadre pas. *Piste d'amélioration* : appliquer le même pattern `assertAccess` que dans `MessagingService`.
- **Reprise après incident non automatisée.** L'état d'une campagne email interrompue par un redémarrage serveur (destinataires encore `PENDING`) survit en base de données, mais aucun mécanisme planifié ne relance automatiquement l'envoi — une reprise manuelle est nécessaire. *Piste d'amélioration* : un `@Scheduled` au démarrage de l'application qui détecte les campagnes `SENDING` avec des destinataires `PENDING` et relance `sendAsync()`.

3.4 Conclusion

Ce chapitre a détaillé, par ordre d'impact, les mécanismes qui distinguent IAT Academy d'une simple application CRUD : intégration IA résiliente, correction hybride avec réconciliation d'état, progression métier en cascade, contrôle d'accès à granularité fine, et les limites identifiées en toute transparence. Le chapitre 4 présente maintenant la démarche de validation et de déploiement.

Chapitre 4

Tests et déploiement

4.1 Stratégie de tests

La qualité du backend est vérifiée par une suite de tests unitaires et de service exécutée avec JUnit 5 et Mockito. Aucun contexte Spring complet n'est démarré pour ces tests : les dépendances de chaque service sont simulées avec `@Mock` ou `Mockito.spy()`, ce qui rend la suite rapide à exécuter aussi bien en local qu'en intégration continue, et permet de tester en isolation la logique métier sans dépendre d'une base de données réelle. Ce choix a en outre orienté certains choix d'implémentation présentés au chapitre 3 : les méthodes `callGemini` et `callGrok` de `QuestionGenerationService`, par exemple, sont délibérément déclarées *package-private* plutôt que *private*, dans le seul but de pouvoir être simulées par `Mockito.spy()` sans effectuer de véritable appel réseau lors des tests.

4.1.1 Couverture côté backend

TABLE 4.1 – Classes de test backend (backend/src/test/java/ma/iatacademy/api/)

Classe de test	Objet couvert
QuizScoringLogicTest	Logique de correction automatique des quiz
SecurityRbacTest	Contrôle d'accès par rôle (RBAC)
AgendaServiceTest	Planification / agenda
AssignmentServiceTest	Devoirs
AuthServiceTest	Authentification, inscription, mot de passe
BadgeServiceTest	Gamification (badges)
CertificateServiceTest	Délivrance et vérification de certificats
CommentServiceTest	Commentaires de leçon
CourseAssistantServiceTest	Assistant de cours (IA)
EarlyWarningServiceTest	Détection d'inactivité
GradebookServiceTest	Carnet de notes
GroupServiceTest	Groupes d'apprenants
JobOfferServiceTest	Bourse à l'emploi (accès diplômés)
LessonLockServiceTest	Verrou d'édition collaboratif
LessonNoteServiceTest	Notes personnelles de leçon
MediaServiceTest	Gestion des médias / extraction PDF
MessagingServiceTest	Messagerie interne
ProgressionServiceTest	Déblocage séquentiel du parcours
QuestionGenerationServiceTest	Génération de questions par IA
RateLimitServiceTest	Limitation de débit Redis
SearchServiceTest	Recherche globale
StageServiceTest	Gestion du stage
UfValidationServiceTest	Validation d'unité de formation

Au total, cette suite comporte **138 méthodes de test** (@Test), couvrant en priorité les services identifiés au chapitre 3 comme porteurs de la logique métier la plus sensible : notation, sécurité, validation d'UF, intégration à des services d'intelligence artificielle externes. Cette priorisation n'est pas anodine : un service comme `ProgressionServiceTest`, par exemple, teste explicitement les deux branches distinctes du calcul de progression — apprenant individuel suivant la règle séquentielle automatique par UF, et apprenant rattaché à un groupe suivant les assignations manuelles du Directeur — puisqu'une régression sur l'une de ces deux branches serait particulièrement difficile à détecter par simple observation manuelle de l'application, la seconde branche ne concernant qu'une catégorie précise d'apprenants.

4.1.2 Couverture côté frontend et tests de charge

TABLE 4.2 – Tests frontend

Type	Outil	Fichiers
Tests unitaires / composants	Vitest + React Testing Library	QuizTimer.test.tsx, QuizAttemptReview
Tests de bout en bout	Playwright	auth.spec.ts, quiz.spec.ts

Le projet comprend enfin un dossier `loadtest/` dédié aux tests de montée en charge, écrits comme des scripts autonomes plutôt qu'intégrés à la suite JUnit, dans la mesure où leur objet n'est pas de vérifier une règle métier mais un comportement sous concurrence. Le script `quiz-start-concurrency.js` vérifie précisément le point discuté au chapitre 3 (§3.2.2) : que le verrou consultatif Postgres (`pg_advisory_xact_lock`) empêche effectivement la création de deux tentatives de quiz simultanées pour un même couple utilisateur/quiz lorsque plusieurs requêtes de démarrage arrivent en parallèle. Le script `login-rate-limit.js` vérifie de son côté que le mécanisme de limitation de débit décrit au §3.2.5 rejette effectivement les tentatives de connexion au-delà du seuil configuré, et `read-throughput.js` mesure le débit de lecture soutenu par l'API sur les endpoints les plus consultés. Ces scripts, bien que ne remplaçant pas une campagne de tests de charge outillée (par exemple avec `k6` ou `Gatling`), ont permis de valider empiriquement — et non uniquement par lecture du code — que les mécanismes de concurrence et de limitation de débit se comportaient comme attendu sous une charge simultanée réaliste.

4.2 Architecture de déploiement

La Figure 4.1 présente la topologie de déploiement du projet. À l'heure actuelle, le fichier `docker-compose.yml` placé à la racine du dépôt active uniquement les services PostgreSQL 16 et Redis 7, respectivement exposés sur les ports hôtes 5433 et 6379 et adossés chacun à un volume nommé (`postgres_data`, `redis_data`) garantissant la persistance des données entre redémarrages de conteneur ; chaque service embarque en outre un test de santé (`healthcheck`) — `pg_isready` pour PostgreSQL, `redis-cli ping` pour Redis — permettant à d'éventuels services dépendants d'attendre leur disponibilité réelle avant de démarrer. Les services applicatifs (API Spring Boot et frontend Next.js) sont pour leur part déjà décrits, à l'état de commentaire dans ce même fichier, en attente d'une activation ultérieure une fois la configuration de production stabilisée.

Cette topologie cible n'est cependant pas une simple hypothèse de travail : elle est déjà entièrement anticipée par la configuration `nginx` du projet (`deploy/nginx/conf.d/elearning.conf`), qui définit deux *upstreams* nommés — `iat_api` pointant vers le conteneur API sur le port 8080, et `iat_frontend` pointant vers le conteneur frontend sur le port 3000 — et route les requêtes entrantes en conséquence : les chemins `/api/`, `/swagger-ui/` et `/api-docs` sont dirigés vers

l'API (avec un délai d'attente étendu à 300 secondes pour tolérer les opérations longues), tandis que toute autre requête, y compris les mises à niveau WebSocket, est dirigée vers le frontend. Cette configuration fixe également une taille maximale de corps de requête de 520 Mo, dimensionnée pour absorber les imports de médias volumineux (vidéos, PDF) traités par la plateforme. Le backend lui-même est packagé, via `backend/Dockerfile`, selon un *build* Docker en deux étapes : une première image de compilation (`eclipse-temurin:17-jdk`) qui télécharge les dépendances Maven puis compile l'application, et une seconde image d'exécution, nettement plus légère (`eclipse-temurin:17-jre`), qui ne conserve que l'archive exécutable finale — une pratique standard qui réduit sensiblement la taille de l'image livrée en production par rapport à une image de compilation unique.

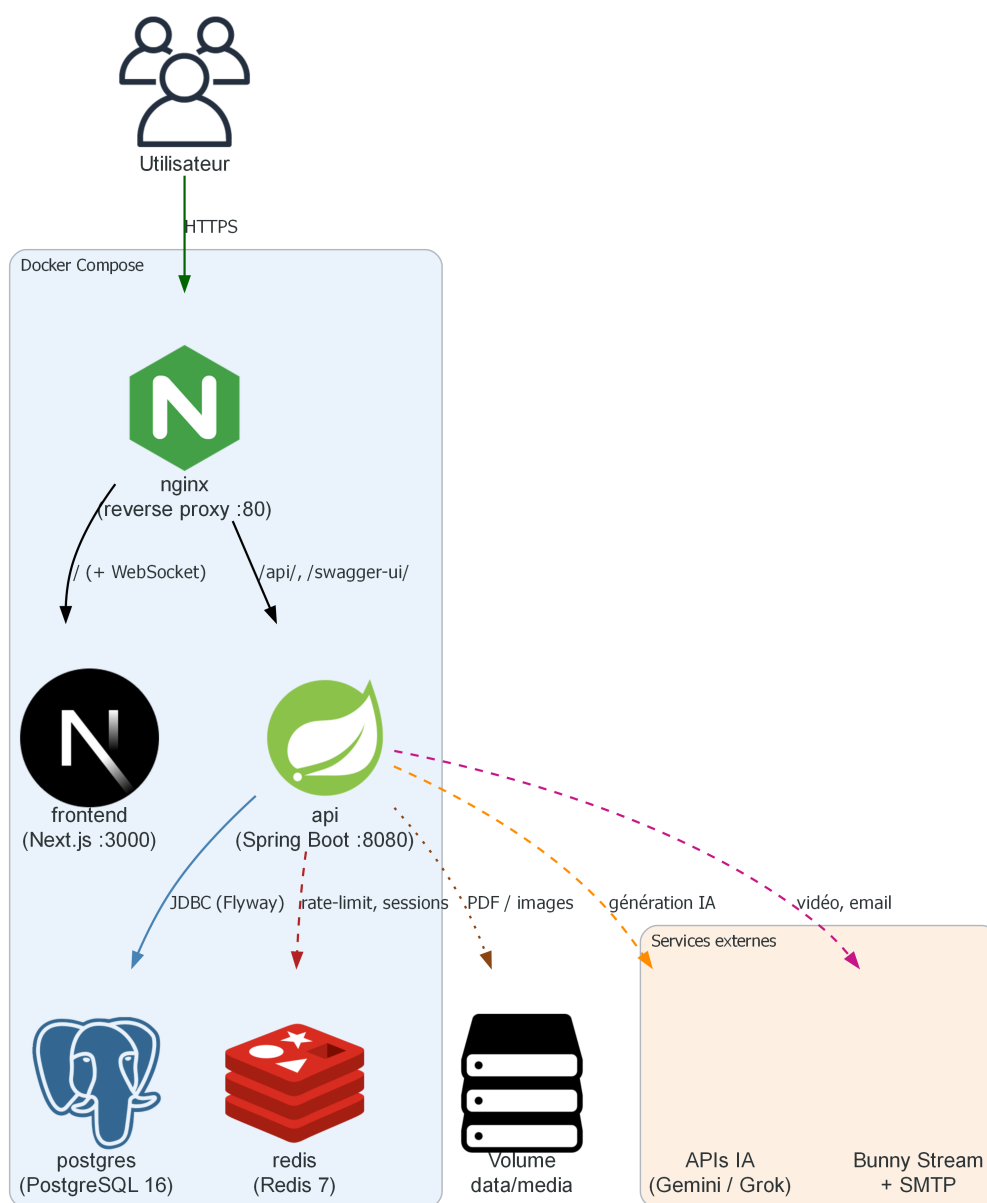


Diagramme de déploiement - IAT Academy

FIGURE 4.1 – Architecture de déploiement

Sur le plan de la configuration, l'ensemble des paramètres sensibles — identifiants de connexion à la base de données, clé de signature des jetons JWT, clés d'API des fournisseurs d'intelligence artificielle externes — est injecté exclusivement par variables d'environnement (fichier `.env`, jamais versionné dans le dépôt; voir `.env.example` pour la liste complète des clés attendues et leur rôle), conformément au principe des *Twelve-Factor Apps* qui veut qu'une même image d'application puisse être déployée sur des environnements différents sans qu'aucun secret ne soit jamais gravé dans le code ou dans l'image elle-même. Ce même mécanisme est celui qui permet, par exemple, d'activer ou de désactiver silencieusement l'intégration à Gemini et à Grok

décrite au §3.2.1 : un fournisseur est considéré comme configuré dès lors que sa clé d'API n'est pas vide, sans qu'aucun indicateur booléen supplémentaire ne soit nécessaire.

4.3 Conclusion

Ce chapitre a présenté la démarche de validation du projet — une suite de 138 tests backend complétée par des tests frontend et des scripts de montée en charge ciblés sur les mécanismes de concurrence les plus sensibles — ainsi que l'architecture de déploiement retenue, dont la topologie cible est déjà anticipée par la configuration nginx du projet en attendant son activation complète. La conclusion générale de ce rapport revient à présent sur l'ensemble du travail réalisé durant ce stage.

Conclusion générale

Ce projet de fin d'études a permis de concevoir et de développer IAT Academy, une plateforme e-learning complète destinée aux métiers de l'aviation, de l'accueil et du tourisme, en couvrant l'ensemble du cycle de vie d'un projet logiciel : analyse des besoins, modélisation d'un domaine métier de 37 entités, conception d'une architecture en couches robuste, implémentation d'une API REST sécurisée de 29 contrôleurs, puis validation par une suite de 138 tests automatisés et une démarche de déploiement conteneurisé. Au-delà de la couverture fonctionnelle — catalogue de cours, système de quiz à quatre portées, gestion de stage, certification — ce travail a surtout été l'occasion d'aller au-delà du simple CRUD sur plusieurs points précis, détaillés au chapitre 3 : l'intégration résiliente d'un service d'intelligence artificielle générative avec repli automatique entre deux fournisseurs, la réconciliation d'état entre correction automatique et correction manuelle d'un même quiz, une logique de progression pédagogique en cascade jusqu'à l'éligibilité au certificat, et un contrôle d'accès à granularité fine qui dépasse le simple contrôle par rôle.

Ce travail n'a pas été mené sans difficultés. La principale d'entre elles a consisté à traduire des règles métier parfois implicites — notamment la distinction entre les UF qui exigent une validation manuelle du Directeur et celles qui n'en exigent pas, ou encore la coexistence de deux stratégies de déblocage du contenu selon qu'un apprenant suit son parcours en ligne de façon autonome ou qu'il appartient à une cohorte encadrée — en un modèle de données et une logique applicative à la fois simples et fidèles à la réalité du terrain. La confrontation régulière avec l'équipe pédagogique d'IAT Academy a été déterminante pour éviter qu'une simplification excessive du modèle ne finisse par trahir ces règles. Le chapitre 3 assume également, en toute transparence, deux limites concrètes identifiées dans le code existant — une incohérence dans l'application du contrôle de propriété au niveau du service de gestion des devoirs, et l'absence d'une reprise automatique après l'interruption d'une campagne email — qui constituent naturellement les premières pistes d'amélioration à court terme.

À plus long terme, plusieurs évolutions permettraient d'enrichir encore la plateforme : l'activation complète de la topologie de déploiement déjà anticipée par la configuration nginx (services API et frontend conteneurisés aux côtés de PostgreSQL et Redis), la mise en place d'une intégration continue automatisée pour exécuter la suite de tests à chaque modification, ou encore l'extension du mécanisme de génération de questions par intelligence artificielle à d'autres formats pédagogiques. Au-delà de l'aspect technique, ce stage a été une expérience profession-

nelle formatrice à part entière : il a permis de mettre en pratique, sur un projet réel destiné à des utilisateurs réels, des compétences d'analyse, de conception et de rigueur technique qui dépassent largement le cadre d'un exercice académique classique.

Bibliographie

- [1] « OWASP Top 10, » OWASP Foundation, visité le 1^{er} jan. 2026. adresse : <https://owasp.org/www-project-top-ten/>